

Dědičnost v OOP

Abstrakce, rozhraní, polymorfismus

18

Maturitní otázka č. 18

Proč vůbec dědičnost?



"Pes JE zvíře"(doufejme)

Dědičnost modeluje vztah is-a mezi třídami a umožňuje sdílet společný kód mezi potomky.



DRY princip

Společný kód píšeš jen jednou — v rodičovské třídě.



Hierarchie

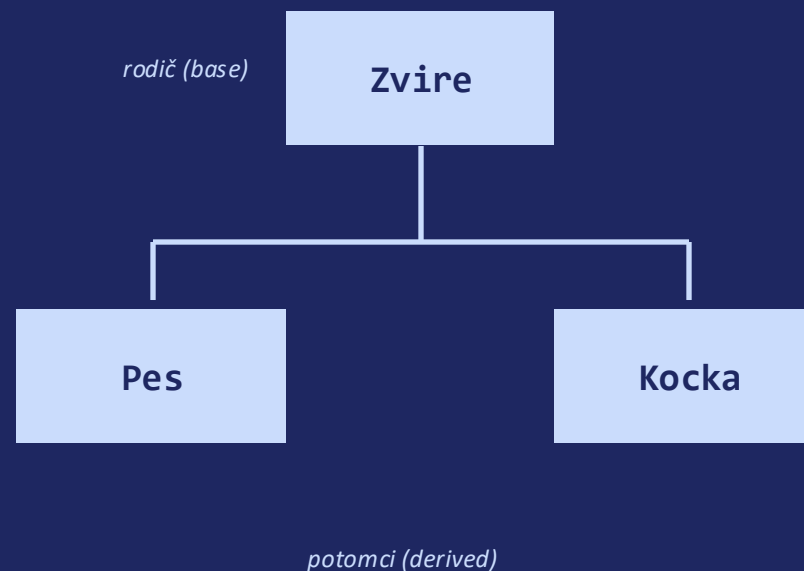
Logická struktura: obecné → konkrétní.



Polymorfismus

Jedna kolekce, různé typy potomků.

Příklad hierarchie



C# syntax: `class Pes : Zvire { ... }`

Základní syntax dědičnosti



Potomek dědí všechny public a protected členy rodičovské třídy. V C# se dědičnost zapisuje dvojtečkou za jménem třídy.

```
// Rodičovská třída
class Zvire
{
    public string Jmeno { get; set; }
    public void Jist() {
        Console.WriteLine($"{Jmeno} jí.");
    }
}

// Potomek – dědí pomocí ":"
class Pes : Zvire
{
    public void Stekat() {
        Console.WriteLine("Haf!");
    }
}
```

Co se děje?

- Pes má přístup k Jmeno a Jist() automaticky
- Přidává vlastní metodu Stekat()
- C# podporuje pouze jednoduchou dědičnost (jeden rodič)
- Více schopností řeší přes rozhraní (interface)

Použití:

```
Pes rex = new Pes { Jmeno = "Rex" };
rex.Jist();      // z rodiče
rex.Stekat();    // vlastní
```

Abstraktní třída



Šablona pro potomky, která SAMA o sobě nemůže existovat.

Klíčové vlastnosti:

- Označuje se klíčovým slovem `abstract`
- Nelze vytvořit instanci přes `new`
- Může mít běžné i abstraktní metody
- Potomek MUSÍ implementovat všechny abstraktní metody



Kdy použít: Skupina podobných tříd se společným chováním, ale "obecnou" třídu nechceš instancovat.

```
abstract class Zvire
{
    public string Jmeno { get; set; }

    // Běžná metoda – má tělo
    public void Jist() { ... }

    // Abstraktní – bez implementace
    public abstract void VydejZvuk();
}

class Pes : Zvire
{
    public override void VydejZvuk()
    {
        Console.WriteLine("Haf!");
    }
}

// Zvire z = new Zvire(); ← CHYBA!
Zvire pes = new Pes();      // ✓ OK
```

Abstraktní vs virtuální metoda



abstract

BEZ implementace
Potomek MUSÍ přepsat
Jen v abstract class

```
abstract class Tvar
{
    public abstract double Obsah();
}

class Kruh : Tvar
{
    public override double Obsah()
    {
        return Math.PI * r * r;
    }
}
```

virtual

S implementací (výchozí)
Potomek MŮŽE přepsat
V běžných třídách

```
class Zvire
{
    public virtual void VydejZvuk()
    { WriteLine("..."); }
}

class Pes : Zvire
{
    public override void VydejZvuk()
    { WriteLine("Haf!"); }
}

// Kocka nepřepíše → výchozí
```

Override a volání rodiče: base



Kdy se používá override?

abstract metoda **POVINNÉ**

virtual metoda **VOLITELNÉ**

interface metoda **BEZ override**

běžná metoda **NELZE**

sealed override

Zakáže dalším potomkům metodu přepisovat. Užitečné když chceš "uzavřít" implementaci.

Volání rodičovské metody — base

```
class Zamestnanec
{
    public virtual void Pracuj()
    { WriteLine("Pracuji 8h"); }
}

class Manager : Zamestnanec
{
    public override void Pracuj()
    {
        base.Pracuj(); // rodič první
        WriteLine("+ řídím tým");
    }
}
```

Výstup:

```
Pracuji 8h
+ řídím tým
```

Rozhraní (interface)



Smlouva o tom, co třída **UMÍ** — *ne co je*.

- Pouze signatury metod a vlastností (bez implementace)*
- Třída může implementovat VÍCE rozhraní
- Konvence pojmenování: prefix I (IBezec, IPlavec)
- Implementace BEZ klíčového slova override
- Vztah CAN-DO (Kachna UMÍ plavat)

*od C# 8.0 mohou mít default implementaci

```
interface IBezec
{ void Behat(); }

interface IPlavec
{ void Plavat(); }

// Více rozhraní najednou
class Kachna : IBezec, IPlavec
{
    public void Behat()
    { WriteLine("chodí"); }

    public void Plavat()
    { WriteLine("plave"); }
}
```

Abstract class vs Interface



Kritérium	Abstract class	Interface
Klíčové slovo	abstract class	interface
Implementace metod	Ano (i abstraktní)	Jen signatury
Počet rodičů	Jen 1 třída	Více rozhraní současně
Použití override	Ano (u abstract/virtual)	Ne
Vztah	IS-A (Pes JE Zvíře)	CAN-DO (Pes UMÍ běhat)
Pole a konstruktor	Ano	Ne



Jak se rozhodnout?

Sdílený **kód** a **stav** → abstract class. Sdílená **schopnost** napříč nesouvisejícími třídami → interface.

Modifikátory přístupu



Modifikátor	Třída	Potomek	Zvenku
public	✓	✓	✓
protected	✓	✓	X
private	✓	X	X
internal	✓	✓*	✓*

* ve stejném projektu (assembly)

protected — klíč k dědičnosti

Kompromis mezi private (skrytý) a public (veřejný). Umožňuje sdílet implementační detaily s potomky, ale nepouštět je ven z hierarchie.

```
class BankovniUcet
{
    protected decimal zustatek;
    private string pin;
}

class SporiciUcet : BankovniUcet
{
    public void PripoctiUrok()
    {
        zustatek *= 1.02m; // ✓
        // pin = "..."; ← CHYBA
    }
}

var u = new SporiciUcet();
// u.zustatek = 0; ← zvenku nejde
```

Dědění konstruktorů: base(...)



Konstruktory se NEDĚDÍ! Potomek musí definovat vlastní konstruktor a typicky volá rodičovský přes : **base(...)**

```
class Zvire
{
    public Zvire(string jmeno, int vek)
    {
        WriteLine("1. Konstruktor Zvire");
        Jmeno = jmeno; Vek = vek;
    }
}

class Pes : Zvire
{
    public Pes(string j, int v, string p) : base(j, v)
    {
        WriteLine("2. Konstruktor Pes");
        Plemeno = p;
    }
}
```

Pořadí volání

- 1 Rodič**
Konstruktor Zvire
- 2 Potomek**
Konstruktor Pes

Výstup pro new Pes("Rex", 5, "Ovčák"):

1. Konstruktor Zvire
2. Konstruktor Pes

Polymorfismus



"Mnoho tvarů" — jeden objekt, více podob.



Proměnná rodiče, objekt potomka

Zvire z = new Pes(); ✓



Volá se metoda potomka

Runtime polymorphism – rozhoduje override



Jednotné zpracování

List<Zvire> obsahuje Psa i Kočku



Chyták

Bez override se metoda potomka NEVOLÁ — spustí se verze rodiče. Polymorfismus přestává fungovat a je to klasická chyba u zkoušky.

```
List<Zvire> zvirata = new()
{
    new Pes    { Jmeno = "Rex" },
    new Kocka { Jmeno = "Micka" }
};

foreach (Zvire z in zvirata)
{
    Console.WriteLine($"{z.Jmeno}: ");
    z.VydejZvuk(); // každý jinak
}

// Výstup:
// Rex: Haf!
// Micka: Mňau!
```

Shrnutí a maturitní chytáky

Co si pamatovat

- Dědičnost = is-a vztah + sdílení kódu
- abstract class — nelze instancovat, vynucuje implementaci
- interface — více dědičnost schopností (can-do)
- virtual MŮŽE, abstract MUSÍ přepsat
- override jen u abstract/virtual, ne u interface
- Konstruktory se nedědí — používá se : base(...)
- Polymorfismus = jedna proměnná, více chování

Na co si dát pozor

Abstract class

Nelze new Tvar() — kompilátor to neprojde

Interface

Metody implementovat BEZ override

Konstruktory

Nedědí se — pokud má rodič parametrický, potomek MUSÍ : base(...)

Chybějící override

Polymorfismus se tiše vypne

Více tříd

C# zakazuje — více rodičů řeší interface